

FINITE ELEMENTS IN PRACTICE · PART I

Introduction to FEniCS

From weak forms to working code

Daniel Fernandez

Weak form $\longrightarrow$ Python code $\longrightarrow$ verified solution

FEniCS installation

Conda (Linux/macOS):

```
1 conda create -n fenicsx-env
2 conda activate fenicsx-env
3 conda install -c conda-forge fenics-dolfinx mpich pyvista
4 conda activate fenicsx-env
```

From the weak form to a solver

Once you have the weak formulation, the implementation needs six steps:

1. Write a mesh generator
2. Implement basis functions
3. Assemble stiffness/mass matrices
4. Handle boundary conditions
5. Solve the linear system
6. Post-process and visualize

FEniCS provides mesh and element tools and assembles the system from a weak form written in Python.

The weak form in code

On paper:

Find $u \in V$ such that

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx \quad \forall v \in \hat{V}$$

In FEniCS:

```
1 u = TrialFunction(V)
2 v = TestFunction(V)
3
4 a = dot(grad(u), grad(v)) * dx
5 L = f * v * dx
6
7 solve(a == L, u_h, bc)
```

The two integrals translate directly into UFL expressions.

Strength: Write the weak form and let FEniCS assemble it.

Trade-off: Custom assembly requires working below this interface.

Note: hp -adaptivity is not directly “implemented” in FEniCS.

A brief FEM recap

Given a PDE: $-\Delta u = f$ in Ω , $u = g$ on $\partial\Omega$.

1. **Weak form:** Find $u \in H_g^1(\Omega)$ s.t. $a(u, v) = L(v) \forall v \in H_0^1(\Omega)$

$$a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v \, dx, \quad L(v) = \int_{\Omega} f v \, dx$$

2. **Discretize:** Choose finite-dimensional $V_h^g \subset H_g^1(\Omega)$ and $V_{h,0} \subset H_0^1(\Omega)$ (piecewise polynomials on a mesh $\mathcal{T}_h$), and write $u_h = g_h + w_h$ with $w_h \in V_{h,0}$. Dirichlet data is imposed by interpolating g on boundary DOFs.
3. **Assemble:** Build stiffness matrix $A_{ij} = a(\phi_j, \phi_i)$ and load vector $b_i = L(\phi_i)$
4. **Solve:** $Au = b$

You choose the mesh, elements, and solver; FEniCS assembles the symbolic forms.

Math objects $\rightarrow$ FEniCS objects

Mathematics	FEniCS class	Code
Mesh $\mathcal{T}_h$	Mesh	<code>UnitSquareMesh(n, n)</code>
Space V_h	FunctionSpace	<code>FunctionSpace(mesh, 'P', k)</code>
Trial function u	TrialFunction	<code>u = TrialFunction(V)</code>
Test function v	TestFunction	<code>v = TestFunction(V)</code>
Solution u_h	Function	<code>u_h = Function(V)</code>
g (boundary data)	Expression	<code>Expression('x[0]*x[0]', ...)</code>
$a(u, v)$	UFL form	<code>dot(grad(u), grad(v))*dx</code>
$L(v)$	UFL form	<code>f*v*dx</code>
BC: $u _{\partial\Omega} = g$	DirichletBC	<code>DirichletBC(V, g, bdry)</code>

EXAMPLE 1

Poisson equation with a known solution



Problem statement

Poisson equation with known exact solution

$$\begin{aligned} -\Delta u &= -6 \quad \text{in } \Omega = [0, 1]^2 \\ u &= 1 + x^2 + 2y^2 \quad \text{on } \partial\Omega \end{aligned}$$

Exact solution: $u(x, y) = 1 + x^2 + 2y^2$.

Why this problem?

- ▶ Polynomial exact solution $\Rightarrow$ we can verify convergence rates
- ▶ P_2 elements should give machine-precision accuracy (why?)

Step 1: mesh and function space

FEniCS

```
1 from fenics import *
2
3 mesh = UnitSquareMesh(8, 8)
4 V = FunctionSpace(mesh, 'P', 1)
```

FEniCSx

```
1 from mpi4py import MPI
2 from dolfinx import fem, mesh
3
4 msh = mesh.create_unit_square(
5     MPI.COMM_WORLD, 8, 8)
6 V = fem.functionspace(
7     msh, ("Lagrange", 1))
```

- ▶ Same mathematical setup, with updated object names and constructors in FEniCSx.
- ▶ Discretization choice: ("Lagrange", k) means P_k elements (e.g. P_1 linear, P_2 quadratic); DG and mixed spaces are also available.
- ▶ Other mesh choices: UnitCubeMesh, RectangleMesh, BoxMesh, unstructured meshes from Gmsh, and imported XDMF/XML meshes.

Step 2: boundary conditions

FEniCS

```
1 u_D = Expression(  
2     '1 + x[0]*x[0] + 2*x[1]*x[1]',  
3     degree=2)  
4  
5 def boundary(x, on_boundary):  
6     return on_boundary  
7  
8 bc = DirichletBC(V, u_D, boundary)
```

FEniCSx

```
1 u_D = fem.Function(V)  
2 u_D.interpolate(  
3     lambda x: 1 + x[0]*x[0]  
4     + 2*x[1]*x[1])  
5  
6 tdim = msh.topology.dim  
7 fdim = tdim - 1  
8 msh.topology.create_connectivity(  
9     fdim, tdim)  
10 boundary_facets = mesh.exterior_  
11 facet_indices(msh.topology)  
12 boundary_dofs = fem.locate_dofs_  
13 topological(V, fdim, boundary_facets)  
14 bc = fem.dirichletbc(u_D, boundary_dofs)
```

- ▶ Neumann BC enters the weak form as a boundary load, e.g. $+ g*v*ds$.
- ▶ Robin BC splits into bilinear/linear parts, e.g. $+ \beta u*v*ds$ and $+ \beta u_R*v*ds$.
- ▶ Use marked boundary measures: `Measure('ds', subdomain_data=...)` (FEniCS) or `ufl.Measure("ds", domain=msh, subdomain_data=...)` (FEniCSx).

Step 3: variational problem

FEniCS

```
1 u = TrialFunction(V)
2 v = TestFunction(V)
3 f = Constant(-6.0)
4
5 a = dot(grad(u), grad(v))*dx
6 L = f*v*dx
```

FEniCSx

```
1 import ufl
2 from dolfinx import default_scalar_type
3
4 u = ufl.TrialFunction(V)
5 v = ufl.TestFunction(V)
6 f = fem.Constant(
7     msh, default_scalar_type(-6.0))
8
9 a = ufl.dot(ufl.grad(u),
10            ufl.grad(v))*ufl.dx
11 L = f*v*ufl.dx
```

Step 4: solve and analyze

FEniCS

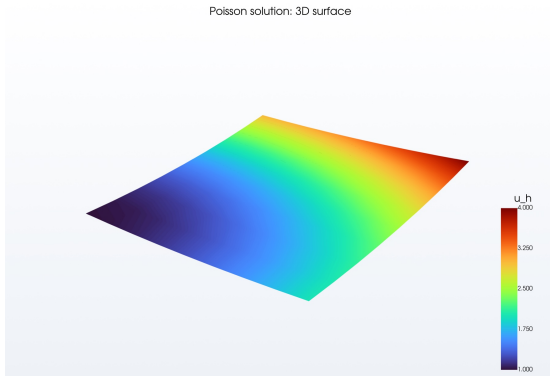
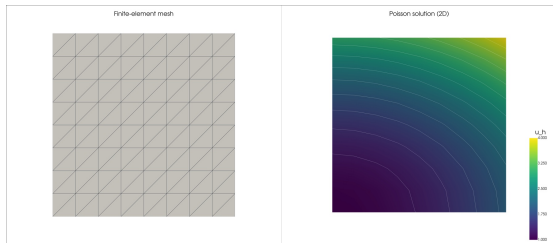
```
1 u_h = Function(V)
2 solve(a == L, u_h, bc)
3
4 error_L2 = errornorm(u_D, u_h, 'L2')
5 error_H1 = errornorm(u_D, u_h, 'H1')
6 print(f"L2 error: {error_L2:.6e}")
7 print(f"H1 error: {error_H1:.6e}")
8
9 plot(u_h)
10 plot(mesh)
```

FEniCSx

```
1 from dolfinx.fem.petsc import LinearProblem
2
3 problem = LinearProblem(
4     a, L, bcs=[bc],
5     petsc_options_prefix="exp1_",
6     petsc_options={"ksp_type": "preonly",
7                    "pc_type": "lu"})
8 u_h = problem.solve()
9
10 err = fem.form(ufl.inner(u_h-u_D, u_h-u_D)
11                * ufl.dx)
12 E_L2 = np.sqrt(msh.comm.allreduce(
13     fem.assemble_scalar(err), op=MPI.SUM))
14 print(f"L2 error: {E_L2:.6e}")
```

- ▶ Default: legacy `solve(...)` uses backend defaults (often LU); FEniCSx uses PETSc defaults unless `petsc_options` are set.
- ▶ Solver families: CG/GMRES/MINRES with ILU/HYPRE/GAMG/MUMPS, plus Newton/SNES for nonlinear systems.
- ▶ In inverse settings, add regularization, e.g. Tikhonov $\alpha \|m\|^2$ or gradient $\alpha \|\nabla m\|^2$.

Solution and mesh visualization

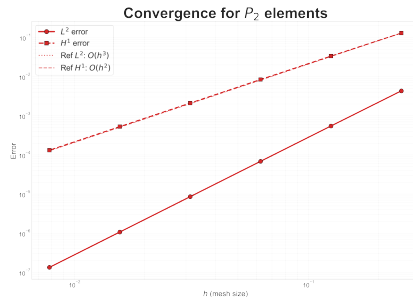
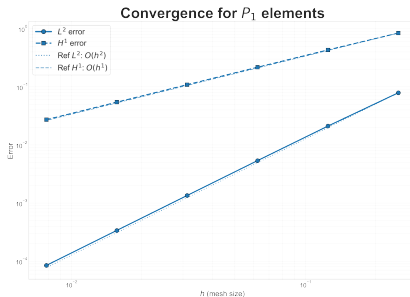


The computed Poisson solution shown as a colour map and a surface.

Convergence study

Solve on meshes
 $N = 4, 8, 16, 32, 64, 128$.

N	h	$\ e\ _{L^2}$	$\ e\ _{H^1}$
4	0.354	5.27e-02	6.01e-01
8	0.177	1.33e-02	3.02e-01
16	0.088	3.33e-03	1.51e-01
32	0.044	8.33e-04	7.56e-02
64	0.022	2.08e-04	3.78e-02
128	0.011	5.21e-05	1.89e-02



P_k elements: $\|u - u_h\|_{L^2} = O(h^{k+1})$, $\|u - u_h\|_{H^1} = O(h^k)$.

These convergence plots (left: P_1 , right: P_2) confirm the expected rates.

EXAMPLE 2

Neumann problem with local mesh refinement



Elliptic example: Neumann BC + local refinement

- ▶ PDE on an L-shaped domain: $-\Delta u + u = f$ in Ω , with homogeneous Neumann BC $\partial_n u = 0$ on $\partial\Omega$.
- ▶ Explicit forcing: $f(x, y) = \frac{1}{(x^2 + y^2 + \varepsilon^2)^{3/4}}$, with $\varepsilon = 6 \times 10^{-3}$.
- ▶ This forcing is singular near the re-entrant corner, so local mesh refinement near $(0, 0)$ gives better accuracy per DOF.
- ▶ The script compares uniform refinement with refinement near the corner.

Weak formulation ($V = H^1(\Omega)$): find $u \in V$ such that

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx + \int_{\Omega} u v \, dx = \int_{\Omega} f v \, dx \quad \forall v \in V.$$

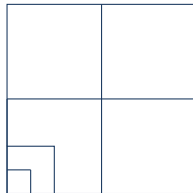
Neumann example: key code

```
1 V = fem.functionspace(msh, ("Lagrange", 1))
2 u = ufl.TrialFunction(V); v = ufl.TestFunction(V)
3 x = ufl.SpatialCoordinate(msh)
4 r2 = x[0]*x[0] + x[1]*x[1] + eps*eps
5 f_expr = 1.0 / (r2**0.75)
6
7 # Pure Neumann: no Dirichlet constraints
8 a = (ufl.inner(ufl.grad(u), ufl.grad(v)) + u*v) * ufl.dx
9 L = f_expr * v * ufl.dx
10 u_h = LinearProblem(a, L, bcs=[]).solve()
11
12 # Corner-focused weighted error metric
13 weight = ufl.exp(-(x[0]*x[0] + x[1]*x[1]) / (0.06**2))
14 E_corner = fem.assemble_scalar(fem.form(weight * u_h * ufl.dx))
```

- ▶ The extra $+u$ term makes the pure-Neumann model uniquely solvable.
- ▶ Neumann data is natural in the weak form, so no boundary DOF constraints are added.
- ▶ A weighted corner error metric near $(0,0)$ quantifies refinement accuracy.

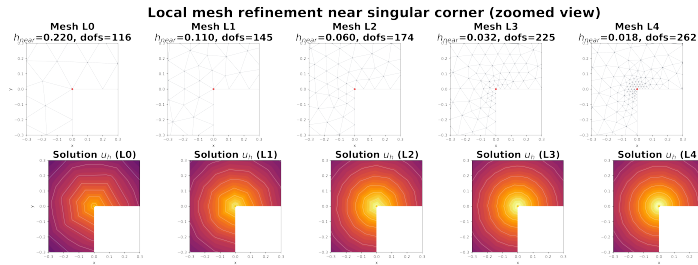
Adaptive mesh refinement (Neumann example)

```
1 # Error indicator
2 e = u_exact - u_h
3 indicators = project(
4     inner(grad(e), grad(e)), DG0)
5
6 # Mark cells with large error
7 cell_markers = MeshFunction(
8     "bool", mesh,
9     mesh.topology().dim(), False)
10
11 # Refine top 30% of cells
12 cutoff = sorted(indicators.vector()[ :],
13     reverse=True)[int(0.3*mesh.num_cells())]
14
15 for c in cells(mesh):
16     if indicators(c.midpoint()) > cutoff:
17         cell_markers[c] = True
18
19 mesh = refine(mesh, cell_markers)
```

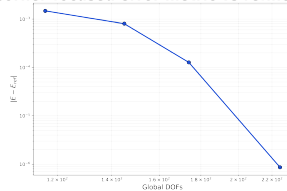


Adaptive refinement near singularity

Neumann example: results



Corner-focused error metric vs refinement



Local refinement near the re-entrant corner improves both field resolution and the corner-weighted error-metric convergence.

EXAMPLE 3

L-shaped Laplacian eigenfunction



Elliptic example: L-shaped Laplacian eigenfunction

- ▶ Eigenproblem on an L-shaped domain: $-\Delta\phi = \lambda\phi$ in Ω , with $\phi = 0$ on $\partial\Omega$.
- ▶ The re-entrant corner induces singular behavior and a characteristic mode shape.
- ▶ Compute the first eigenmode and inspect its behaviour near the corner.

Weak formulation ($V = H_0^1(\Omega)$): find $(\lambda, \phi) \in \mathbb{R} \times V$, $\phi \neq 0$, such that

$$\int_{\Omega} \nabla\phi \cdot \nabla v \, dx = \lambda \int_{\Omega} \phi v \, dx \quad \forall v \in V.$$

Handling $(\lambda, \phi) \in \mathbb{R} \times V$: discretize only ϕ in V_h , obtain $A\Phi = \lambda M\Phi$, and fix the scale with $\Phi^T M\Phi = 1$ (equivalently $\|\phi_h\|_{L^2(\Omega)} = 1$).

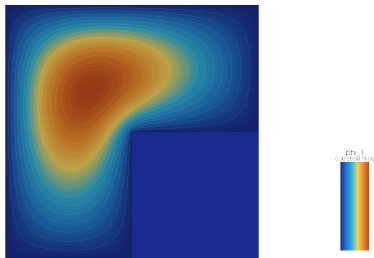
L-shaped eigenproblem: key code

```
1 u = ufl.TrialFunction(V); v = ufl.TestFunction(V)
2 A = assemble_matrix(fem.form(inner(grad(u), grad(v))*dx)); A.assemble()
3 M = assemble_matrix(fem.form(u*v*dx)); M.assemble()
4 A_sp = petsc_to_scipy(A); M_sp = petsc_to_scipy(M)
5
6 # Enforce phi=0 on boundary by removing boundary dofs
7 free_dofs = np.setdiff1d(np.arange(num_dofs), boundary_dofs)
8 A_free = A_sp[free_dofs][:, free_dofs]
9 M_free = M_sp[free_dofs][:, free_dofs]
10
11 eigvals, eigvecs = eigsh(A_free, k=8, M=M_free, sigma=0.0, which="LM")
12 phi = np.zeros(num_dofs)
13 phi[free_dofs] = eigvecs[:, 0]
14 phi /= np.max(np.abs(phi))
```

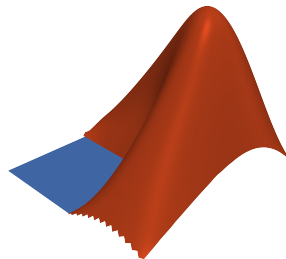
- ▶ This solves the generalized eigenproblem $A\phi = \lambda M\phi$.
- ▶ Boundary elimination strongly enforces homogeneous Dirichlet conditions.
- ▶ Plot the normalized first mode as contours and as a surface.

L-shaped eigenproblem: results

L-shaped domain: first eigenfunction (2D)



L-shaped Laplacian eigenfunction (MATLAB-style 3D)



EXAMPLE 4

Heat equation



Heat equation

Problem

$$\frac{\partial u}{\partial t} - \Delta u = 0 \quad \text{in } \Omega \times (0, T]$$

$$u = 0 \text{ on } \partial\Omega, \quad u(x, 0) = \exp(-25 \|(x - \frac{1}{2}, y - \frac{1}{2})\|^2)$$

Backward Euler time discretization:

$$\frac{u^{n+1} - u^n}{\Delta t} - \Delta u^{n+1} = 0$$

Weak form at each step:

$$\underbrace{(u^{n+1}, v) + \Delta t (\nabla u^{n+1}, \nabla v)}_{a(u^{n+1}, v)} = \underbrace{(u^n, v)}_{L(v)}$$

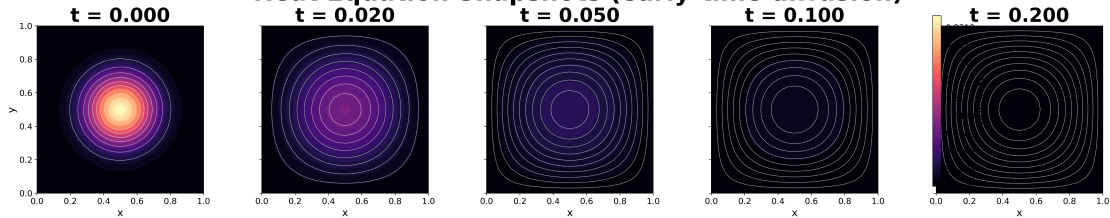
Heat equation: code

```
1 dt = T / num_steps
2 mesh = UnitSquareMesh(30, 30)
3 V = FunctionSpace(mesh, 'P', 1)
4 bc = DirichletBC(V, Constant(0.0), 'on_boundary')
5
6 # Initial condition: Gaussian bump
7 u_n = interpolate(Expression(
8     'exp(-25*((x[0]-0.5)*(x[0]-0.5)+(x[1]-0.5)*(x[1]-0.5)))',
9     degree=2), V)
10
11 u = TrialFunction(V); v = TestFunction(V)
12
13 # Backward Euler weak form
14 a = u*v*dx + dt*dot(grad(u), grad(v))*dx
15 L = u_n*v*dx
16
17 u_h = Function(V)
18 for n in range(num_steps):
19     solve(a == L, u_h, bc)      # Solve one time step
20     u_n.assign(u_h)            # Update u^n <- u^{n+1}
```

Reuse the matrix: With constant Δt , the bilinear form a stays the same. A solver configured to reuse its factorization can avoid repeating that work.

Heat equation: solution snapshots

Heat Equation Snapshots (early-time diffusion)



The Gaussian bump spreads and decreases in height as heat diffuses.

EXAMPLE 5

Nonlinear Poisson equation



Nonlinear Poisson equation

Problem

$$-\nabla \cdot ((1 + u^2) \nabla u) = f \quad \text{in } \Omega, \quad u = 0 \text{ on } \partial\Omega$$

This is a **quasilinear** PDE — the diffusion coefficient depends on u .

Residual form: Find $u \in V$ such that

$$F(u; v) = \int_{\Omega} (1 + u^2) \nabla u \cdot \nabla v \, dx - \int_{\Omega} f v \, dx = 0 \quad \forall v \in \hat{V}$$

FEniCS solves this with **Newton's method**.

The Jacobian $J = F'(u)$ can be computed *automatically*.

Nonlinear Poisson: code

```
1 # u is a Function (NOT TrialFunction) for
2 # nonlinear problems
3 u = Function(V)
4 v = TestFunction(V)
5
6 # Residual  $F(u; v) = 0$ 
7 F = (1 + u**2)*dot(grad(u), grad(v))*dx \
8     - f*v*dx
9
10 # Automatic Jacobian
11 J = derivative(F, u)
12
13 # Newton solver
14 problem = NonlinearVariationalProblem(F, u, bc, J)
15 solver = NonlinearVariationalSolver(problem)
16 solver.solve()
```

For a nonlinear problem: TrialFunction $\rightarrow$ Function (an unknown to be solved for iteratively).

EXAMPLE 6

Stokes flow in a lid-driven cavity



Stokes equations — lid-driven cavity

Problem

$$\begin{aligned} -\Delta \mathbf{u} + \nabla p &= \mathbf{0} && \text{in } \Omega \\ \nabla \cdot \mathbf{u} &= 0 && \text{in } \Omega \\ \mathbf{u} &= (1, 0) && \text{on top wall} \\ \mathbf{u} &= \mathbf{0} && \text{on other walls} \end{aligned}$$

Mixed weak form: Find $(\mathbf{u}, p) \in V \times Q$ such that

$$(\nabla \mathbf{u}, \nabla \mathbf{v}) - (p, \nabla \cdot \mathbf{v}) - (q, \nabla \cdot \mathbf{u}) = 0$$

for all $(\mathbf{v}, q) \in V \times Q$.

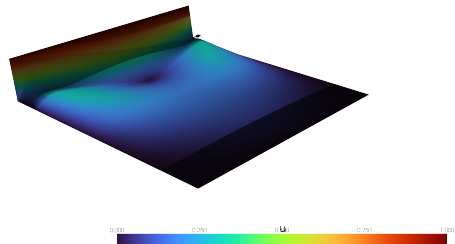
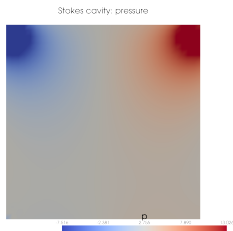
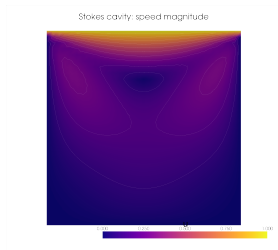
Inf-sup stability: Use Taylor–Hood elements (P_2/P_1) .

Stokes: code

```
1 # Taylor-Hood: P2 velocity, P1 pressure
2 V = VectorFunctionSpace(mesh, 'P', 2)
3 Q = FunctionSpace(mesh, 'P', 1)
4 W = V * Q                                     # Mixed space
5
6 # Boundary conditions
7 bc_lid = DirichletBC(W.sub(0), Constant((1,0)),
8                      'near(x[1],1.0) && on_boundary')
9 bc_walls = DirichletBC(W.sub(0), Constant((0,0)),
10                        '(near(x[0],0)||near(x[0],1)||near(x[1],0)) && on_boundary')
11
12 # Weak form
13 (u, p) = TrialFunctions(W)
14 (v, q) = TestFunctions(W)
15
16 a = (inner(grad(u), grad(v)) - p*div(v) - q*div(u)) * dx
17 L = inner(Constant((0, 0)), v) * dx
18
19 w = Function(W)
20 solve(a == L, w, [bc_lid, bc_walls])
21
22 (u_h, p_h) = w.split() # Extract velocity and pressure
```

Stokes: results

Lid-driven cavity flow (3D)



The computed cavity flow shown in the plane and as a surface.

Official:

- ▶ FEniCS Project: <https://fenicsproject.org>
- ▶ FEniCSx Tutorial: <https://jsdokken.com/dolfinx-tutorial/>
- ▶ Discourse Forum: <https://fenicsproject.discourse.group>

Books & Tutorials:

- ▶ Langtangen & Logg, *Solving PDEs in Python* (Springer, free PDF)
- ▶ The FEniCS Book (Springer, 2012) — comprehensive reference
- ▶ FEniCS Tutorial: <https://fenicsproject.org/tutorial/>

Questions?



Backup: complete Poisson code

```
1 from fenics import *
2 import matplotlib.pyplot as plt
3
4 mesh = UnitSquareMesh(8, 8)
5 V = FunctionSpace(mesh, 'P', 1)
6
7 u_D = Expression('1 + x[0]*x[0] + 2*x[1]*x[1]', degree=2)
8 bc = DirichletBC(V, u_D, 'on_boundary')
9
10 u = TrialFunction(V); v = TestFunction(V)
11 f = Constant(-6.0)
12
13 a = dot(grad(u), grad(v)) * dx
14 L = f * v * dx
15
16 u_h = Function(V)
17 solve(a == L, u_h, bc)
18
19 error_L2 = errornorm(u_D, u_h, 'L2')
20 error_H1 = errornorm(u_D, u_h, 'H1')
21 print(f"L2 error: {error_L2:.6e}")
22 print(f"H1 error: {error_H1:.6e}")
23
24 plot(u_h); plt.colorbar(plot(u_h))
25 plt.savefig("poisson.png", dpi=150)
26 plt.show()
```

Backup: FEniCSx equivalent (DOLFINx)

```
1 import dolfinx, ufl
2 from dolfinx import fem, mesh
3 from mpi4py import MPI
4 import numpy as np
5
6 # Create mesh
7 msh = mesh.create_unit_square(MPI.COMM_WORLD, 8, 8)
8 V = fem.functionspace(msh, ("Lagrange", 1))
9
10 # Boundary condition
11 u_D = fem.Function(V)
12 u_D.interpolate(lambda x: 1 + x[0]**2 + 2*x[1]**2)
13 bdry_facets = mesh.locate_entities_boundary(
14     msh, msh.topology.dim-1, lambda x: np.full(x.shape[1], True))
15 bc = fem.dirichletbc(u_D, fem.locate_dofs_topological(
16     V, msh.topology.dim-1, bdry_facets))
17
18 # Variational problem
19 u = ufl.TrialFunction(V)
20 v = ufl.TestFunction(V)
21 f = fem.Constant(msh, -6.0)
22 a = ufl.dot(ufl.grad(u), ufl.grad(v)) * ufl.dx
23 L = f * v * ufl.dx
24
25 # Solve
26 problem = fem.petsc.LinearProblem(a, L, bcs=[bc])
27 u_h = problem.solve()
```

Same concepts, slightly different API.